# Pertemuan 3
# Representasi Algoritma, List, Dictionary, dan NumPy Array

## Pemrograman untuk Ilmu Data dan Komputasi

MA25-21008 Kelas RA dan RB

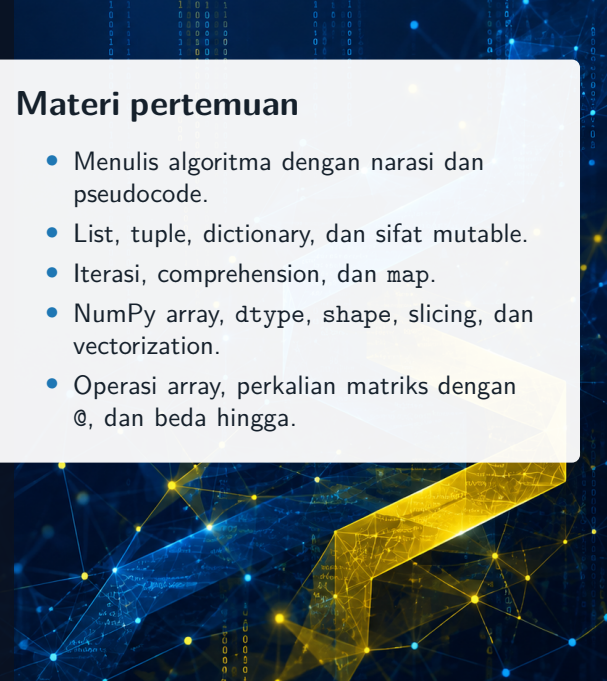
Triyana Muliawati, S.Si., M.Si.
Rifky Fauzi

Program Studi Matematika
Institut Teknologi Sumatera

Semester Ganjil 2026/2027

### Materi pertemuan

- Menulis algoritma dengan narasi dan pseudocode.
- List, tuple, dictionary, dan sifat mutable.
- Iterasi, comprehension, dan `map`.
- NumPy array, `dtype`, `shape`, slicing, dan vectorization.
- Operasi array, perkalian matriks dengan `@`, dan beda hingga.



# Dari satu nilai ke struktur data

## Ketika data masih sedikit

- Satu nilai dapat disimpan dalam satu variabel.
- Rumus dapat ditulis sebagai satu ekspresi.
- Alur program masih mudah dibaca dari atas ke bawah.

## Ketika data mulai banyak

- Kita perlu wadah: list, tuple, dictionary, atau array.
- Kita perlu algoritma: langkah yang dapat ditelusuri.
- Kita perlu verifikasi: cek ukuran, tipe, dan hasil.



# Cara membaca soal komputasi

## Sebelum menulis kode

1. Apa inputnya?
2. Apa output yang diminta?
3. Struktur data apa yang paling cocok?
4. Langkah mana yang berulang?
5. Bagaimana memeriksa hasilnya?

## Sesudah kode berjalan

- Cek tipe data dengan `type` atau `dtype`.
- Cek ukuran data dengan `len` atau `shape`.
- Uji kasus kecil yang dapat dihitung manual.
- Bandingkan versi loop dan NumPy bila relevan.

# Representasi algoritma

| Narasi | Pseudocode | Flowchart |
| --- | --- | --- |
| Kalimat biasa. Cocok untuk menjelaskan ide awal dan asumsi. | Langkah formal yang mirip kode, tetapi tidak terikat sintaks Python. | Gambar alur. Cocok untuk keputusan dan pengulangan. |

Untuk soal LKM, pseudocode membantu memisahkan logika matematika dari detail sintaks.

# Template pseudocode yang dipakai

```
ALGORITMA NamaAlgoritma
INPUT : data masukan dan syaratnya
OUTPUT : keluaran yang diharapkan
BEGIN
    inisialisasi
    langkah-langkah utama
    IF kondisi THEN
        langkah ketika kondisi benar
    ELSE
        langkah ketika kondisi salah
    ENDIF
    FOR / WHILE proses berulang
        perbarui keadaan
    ENDFOR / ENDWHILE
    RETURN hasil
END
```

## Yang harus terlihat

Input, output, kondisi, pengulangan, dan variabel yang berubah dari langkah ke langkah.

# Algoritma Euclid untuk FPB

## Apa yang dicari?

FPB atau *greatest common divisor* (GCD) dari dua bilangan bulat positif adalah bilangan terbesar yang membagi keduanya.

## Ide matematika

Jika

$$a = qb + r, \quad 0 \leq r < b,$$

maka

$$\gcd(a, b) = \gcd(b, r).$$

| $a$ | $b$ | $q$ | $r$ |
|---|---|---|---|
| 252 | 105 | 2 | 42 |
| 105 | 42 | 2 | 21 |
| 42 | 21 | 2 | 0 |

Ketika sisa pembagian menjadi 0, FPB adalah pembagi terakhir: 21.

# Pseudocode dan kode Euclid

```
ALGORITMA Euclid
INPUT : a, b bilangan bulat positif
OUTPUT : gcd(a,b)
BEGIN
    WHILE b != 0
        r = a mod b
        a = b
        b = r
    ENDWHILE
    RETURN a
END
```

```python
def gcd(a, b):
    while b != 0:
        r = a % b
        a = b
        b = r
    return a

print(gcd(252, 105))
```

## Latihan membaca algoritma

Catat pasangan $(a, b)$ setiap kali loop selesai. Itulah jejak eksekusi program.

# Collection di Python

| Struktur | Bentuk | Kegunaan utama |
|---|---|---|
| List | `[3, 5, 8]` | Data berurutan dan dapat diubah. |
| Tuple | `(3, 5, 8)` | Data berurutan dan tetap. |
| Dictionary | `{'A':82}` | Data berpasangan `key:value`. |
| Set | `{3, 5, 8}` | Anggota unik dan operasi himpunan. |
| NumPy array | `np.array([...])` | Komputasi numerik berbasis array. |

Pilihan struktur data menentukan cara iterasi, cara mengubah data, dan jenis operasi yang tersedia.

# Mutable dan immutable

## Mutable

Objek dapat diubah setelah dibuat.

```
a = [1, 2, 3]
a[0] = 99
print(a)
# [99, 2, 3]
```

## Immutable

Objek tidak dapat diubah pada posisi tertentu.

```
b = (1, 2, 3)
b[0] = 99
# TypeError
```

## Mengapa penting?

Saat fungsi menerima list, isi list dapat berubah. Saat menerima tuple, struktur datanya lebih terlindungi.

# List: indeks, slicing, dan list bersarang

```
a = [10, 20, 30, 40, 50]
print(a[0]) # 10
print(a[-1]) # 50
print(a[1:4]) # [20, 30, 40]
print(a[:3]) # [10, 20, 30]
```

```
A = [[1, 2, 3],
     [4, 5, 6],
     [7, 8, 9]]

print(A[1][2]) # 6
print(A[2][0]) # 7
```

## Catatan

List bersarang dapat merepresentasikan matriks, tetapi operasi matematikanya belum otomatis seperti vektor atau matriks.

# List tidak otomatis menjadi vektor

```
>>> [1, 2] + [3, 4]
[1, 2, 3, 4]

>>> 2 * [1, 2]
[1, 2, 1, 2]

>>> [1, 2] * [3, 4]
TypeError
```

## Yang sering disangka

$$[1,2] + [3,4] \stackrel{?}{=} [4,6]$$

$$2[1,2] \stackrel{?}{=} [2,4]$$

## Makna sebenarnya

List adalah wadah berurutan. Untuk operasi vektor numerik, gunakan loop sendiri atau NumPy array.

# Mengolah list dengan loop

```
data = [3, -2, 0, 7, -5, 4, 9]
jumlah_positif = 0
jumlah_kuadrat_negatif = 0
jumlah_nonnol = 0
total_nonnol = 0

for x in data:
    if x > 0:
        jumlah_positif += 1
    if x < 0:
        jumlah_kuadrat_negatif += x**2
    if x != 0:
        jumlah_nonnol += 1
        total_nonnol += x

rata_nonnol = total_nonnol / jumlah_nonnol
```

## Pola umum

Inisialisasi nilai awal, telusuri data satu per satu, perbarui ringkasan.

# Menerapkan fungsi pada list

```python
def f(x):
    return 3*x**2 - 2*x + 1

xs = [-2, -1, 0, 1, 2]

ys = []
for x in xs:
    ys.append(f(x))
```

```python
# list comprehension
ys = [f(x) for x in xs]

# map
ys = list(map(f, xs))

print(ys)
# [17, 6, 1, 2, 9]
```

## Kapan dipakai?

Loop biasa enak untuk langkah panjang. Comprehension ringkas untuk transformasi sederhana. `map` berguna ketika fungsi sudah jelas dan ingin diterapkan ke semua elemen.

# List comprehension sebagai notasi komputasi

Diberikan

$$x = [-2, -1, 0, 1, 2].$$

Hitung

$$y_i = 3x_i^2 - 2x_i + 1, \qquad z_i = x_i y_i.$$

```
x = [-2, -1, 0, 1, 2]
y = [3*t**2 - 2*t + 1 for t in x]
z = [x[i]*y[i] for i in range(len(x))]
total = sum(z)
```

**Yang perlu dibaca**

Comprehension tetap sebuah loop. Bentuknya lebih dekat dengan notasi matematika, tetapi indeks dan panjang data tetap harus diperiksa.

# Dictionary: data berpasangan

```
nilai = {'A': 82, 'B': 71,
         'C': 95, 'D': 64}

for nama, skor in nilai.items():
    print(nama, skor)
```

## Makna

- key: identitas data.
- value: nilai yang tersimpan.
- Cocok untuk tabel kecil berbasis label.

## Contoh penggunaan

Nilai mahasiswa, jumlah kasus per wilayah, parameter model, atau konfigurasi program.

# Transformasi dictionary

```
nilai = {'A':82, 'B':71,
         'C':95, 'D':64}

data = list(nilai.values())
rata = sum(data) / len(data)
var = sum((x-rata)**2 for x in data) / len(data)
s = var**0.5
```

```
z = {}
for nama, x in nilai.items():
    z[nama] = (x - rata) / s

print(z)
```

## Verifikasi

Rata-rata skor $z$ seharusnya mendekati 0. Ini contoh sederhana bahwa program perlu diperiksa dengan sifat matematika.

# Matriks sebagai list of lists

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

```
A = [[1, 2, 3],
     [4, 5, 6],
     [7, 8, 9]]

kolom_akhir = [baris[-1] for baris in A]
diagonal = [A[i][i] for i in range(3)]
kolom_tengah = [2*baris[1]**2 for baris in A]
```

## Perhatikan indeks

Baris dipilih dulu, lalu kolom. Bentuk `A[i][j]` berarti elemen baris ke-`i`, kolom ke-`j`.

# Membentuk matriks dengan loop bersarang

Bentuk matriks $A \in \mathbb{R}^{4\times 4}$ dengan

$$a_{ij} = i + j + ij, \quad i, j = 1, 2, 3, 4.$$

```
A = []
for i in range(1, 5):
    baris = []
    for j in range(1, 5):
        baris.append(i + j + i*j)
    A.append(baris)

print(A)
```

## Struktur algoritma

Loop luar membentuk baris. Loop dalam mengisi kolom. Setelah satu baris selesai, baris dimasukkan ke matriks.

# Algoritma transpose manual

```
ALGORITMA Transpose
INPUT : matriks A berukuran m x n
OUTPUT : matriks B berukuran n x m
BEGIN
    buat B kosong
    FOR j = 0 sampai n-1
        buat baris baru
        FOR i = 0 sampai m-1
            tambahkan A[i][j]
        ENDFOR
        tambahkan baris ke B
    ENDFOR
    RETURN B
END
```

```python
def transpose(A):
    m = len(A)
    n = len(A[0])
    B = []
    for j in range(n):
        baris = []
        for i in range(m):
            baris.append(A[i][j])
        B.append(baris)
    return B
```

# Algoritma perkalian matriks-vektor

Untuk $A \in \mathbb{R}^{m \times n}$ dan $x \in \mathbb{R}^n$,

$$y_i = \sum_{j=1}^{n} a_{ij} x_j.$$

Setiap komponen $y_i$ adalah hasil kali baris ke-$i$ dengan vektor $x$.

```python
def matvec(A, x):
    m = len(A)
    n = len(A[0])
    y = []
    for i in range(m):
        total = 0
        for j in range(n):
            total += A[i][j] * x[j]
        y.append(total)
    return y
```

## Koneksi ke NumPy

Versi manual ini membantu memahami apa yang nanti ditulis ringkas sebagai `A @ x`.

# NumPy array

## Mengapa perlu NumPy?

- Array numerik memiliki tipe data seragam.
- Operasi pada banyak data ditulis lebih ringkas.
- Banyak pustaka ilmiah memakai NumPy sebagai dasar.

```
import numpy as np

a = [0, 0, 1, 4, 7, 16]
b = np.array(a)
c = np.array([1, 4., -2, 7])

print(b.dtype) # int
print(c.dtype) # float
print(c.shape) # (4,)
```

# Membuat array

```
import numpy as np

np.array([1, 2, 3])
np.arange(0, 10, 2)
np.linspace(0, 1, 5)
np.zeros((2, 3))
np.ones((2, 3))
np.eye(3)
```

## Yang perlu dicek

- `shape`: ukuran setiap dimensi.
- `dtype`: tipe data elemen.
- Dimensi: array 1D tidak sama dengan matriks kolom.

## Kebiasaan baik

Gunakan `import numpy as np`. Hindari `from numpy import *`.

# List dan NumPy array: hasil operasi berbeda

```
>>> [1, 2, 3] + [10, 20, 30]
[1, 2, 3, 10, 20, 30]

>>> 2 * [1, 2, 3]
[1, 2, 3, 1, 2, 3]
```

```
>>> import numpy as np
>>> a = np.array([1, 2, 3])
>>> b = np.array([10, 20, 30])
>>> a + b
array([11, 22, 33])
>>> 2 * a
array([2, 4, 6])
```

## Inti

List mengatur urutan objek. NumPy array mendukung operasi numerik seperti vektor.

# Operasi element-wise pada array

```
import numpy as np

x = np.array([-2, -1, 0, 1, 2])
y = 3*x**2 - 2*x + 1
z = x*y

print(y)
print(np.sum(z))
```

## Apa yang terjadi?

- `x**2` menghitung kuadrat tiap elemen.
- `3*x**2 - 2*x + 1` berlaku ke seluruh array.
- `x*y` adalah perkalian elemen dengan elemen.

## Penting

Operasi element-wise bukan perkalian matriks.

# Operator @ untuk perkalian matriks

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 2 & 0 \\ 1 & 5 \end{bmatrix}$$

**Dua operasi berbeda**

`A*B`: perkalian elemen-sejajar.
`A@B`: perkalian matriks.

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[2, 0], [1, 5]])

print(A * B)
# [[ 2 0]
# [ 3 20]]

print(A @ B)
# [[ 4 10]
# [10 20]]
```

# Slicing array

```
import numpy as np

A = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

print(A[0, :]) # baris pertama
print(A[:, 1]) # kolom kedua
print(A[1:, :2]) # submatriks
```

## Perbedaan dengan list

Pada NumPy array dua dimensi, indeks dapat ditulis `A[i,j]`. Pada list of lists, bentuknya `A[i][j]`.

## Kegunaan

Slicing membuat operasi pada bagian data dapat ditulis tanpa loop eksplisit.

# Masking: memilih data dengan kondisi

```
x = np.array([3, -2, 0, 7, -5, 4, 9])
positif = x[x > 0]
genap = x[x % 2 == 0]

print(positif)
print(genap)
```

## Makna

Kondisi `x > 0` menghasilkan array Boolean. Array Boolean itu dipakai untuk mengambil elemen yang memenuhi syarat.

## Koneksi ke data analysis

Masking adalah dasar filtering data sebelum masuk ke pandas.

# Dari turunan ke beda hingga

Dalam kalkulus, turunan didefinisikan sebagai

$$f'(t) = \lim_{h \to 0} \frac{f(t+h) - f(t)}{h}.$$

Jika data hanya tersedia pada titik diskrit, limit tidak dapat dihitung langsung.

Untuk data posisi $y_i$ pada waktu $t_i$, pendekatan kecepatan rata-rata antar titik adalah

$$v_i \approx \frac{y_{i+1} - y_i}{t_{i+1} - t_i}.$$

Ini disebut beda hingga maju pada data diskrit.

Jadi beda hingga bukan rumus yang muncul tiba-tiba. Ia berasal dari ide rasio perubahan pada definisi turunan.

# Beda hingga dengan slicing NumPy

```
import numpy as np

t = np.array([0.0, 0.5, 1.0, 1.5, 2.0])
y = np.array([0.0, 0.2, 0.9, 2.1, 3.8])

v = (y[1:] - y[:-1]) / (t[1:] - t[:-1])

print(v)
print(len(y), len(v))
```

## Mengapa panjangnya berkurang satu?

Setiap nilai $v_i$ memakai dua titik berurutan: titik ke-$i$ dan titik ke-$i+1$. Jika ada $n$ titik data, hanya ada $n-1$ selang antar titik.

# Verifikasi hasil komputasi array

```
y_loop = []
for x in xs:
    y_loop.append(f(x))

y_np = f(np.array(xs))

np.allclose(y_loop, y_np)
```

## Yang diperiksa

- Apakah hasil loop dan NumPy sama?
- Apakah tipe datanya sesuai?
- Apakah ukuran output sesuai harapan?
- Apakah kasus kecil cocok dengan hitungan manual?

# Menyiapkan jawaban untuk tantangan LKM M3




**Contoh pertanyaan sebelum mengerjakan**

Apakah datanya berlabel atau berurutan? Apakah operasi dilakukan per elemen atau antarbaris-kolom? Apakah perlu loop, comprehension, atau NumPy?

# Contoh terpadu: standardisasi data kecil

```
nilai = {'A':82, 'B':71,
         'C':95, 'D':64}

nama = list(nilai.keys())
x = np.array(list(nilai.values()), dtype=float)

z = (x - x.mean()) / x.std()
hasil = dict(zip(nama, z))
```

## Langkah berpikir

1. Dictionary menyimpan label dan nilai.
2. Array dipakai untuk komputasi numerik.
3. Hasil dikembalikan lagi ke dictionary.
4. Rata-rata skor $z$ dicek mendekati nol.

# Kesalahan yang sering muncul

## Pada list dan dictionary

- Mengira list otomatis seperti vektor.
- Salah indeks: mulai dari 1 padahal Python mulai dari 0.
- Lupa menggunakan `.items()` saat butuh key dan value.

## Pada NumPy

- Bingung `*` dan `@`.
- Tidak mengecek `shape`.
- Mengira `dtype` tidak penting.
- Slicing menghasilkan ukuran yang berubah.

# Quiz e-learning M3

## Materi quiz

- Membaca pseudocode dan menelusuri loop.
- GCD dengan algoritma Euclid.
- Mutable dan immutable.
- Operasi list dibandingkan NumPy array.
- Dictionary dan iterasi `items()`.
- `shape`, `dtype`, slicing, dan operator `@`.

## Cara belajar

Prediksi dahulu hasil kode. Setelah itu jalankan Python untuk memeriksa dan menulis koreksi pemahaman.

# Tugas mandiri M3

## Yang dikumpulkan

1. Pseudocode dan implementasi algoritma Euclid.
2. Contoh mutable dan immutable beserta penjelasan.
3. Program transformasi list dengan loop, comprehension, dan `map`.
4. Dictionary nilai dan standardisasi skor.
5. Matriks list of lists: transpose dan perkalian matriks-vektor manual.
6. NumPy array: operasi element-wise, `@`, slicing, dan beda hingga.

## Format

Boleh menggunakan Google Colab, Jupyter, Spyder, VS Code, atau IDE Python lain. Kode harus dapat dijalankan ulang.

# Penutup

## Setelah pertemuan ini

Mahasiswa diharapkan dapat memilih representasi data, menulis algoritma sederhana, dan mengubahnya menjadi kode Python yang dapat diperiksa.

## Menuju pertemuan berikutnya

Kode yang sudah berjalan perlu dibuat lebih aman: exception, debugging, testing, terminasi, dan rekursi akan diperiksa lebih teliti.

## Rujukan

Miller dan Ranum: algoritma dan struktur data. Pine: list, dictionary, array, dan komputasi ilmiah. McKinney: struktur data Python, NumPy array, vectorized computation, dan workflow data.